

Group 10

Natalie Kaplan (njk82), Dina Nabavian (dn333), Cleo Hamilton (chh89)

Robot Design and Strategy Overview

Our robot was built with an 8 inch by 8 inch laser cut square frame to utilize all of the space available in the initial bounding box. The robot had cardboard flaps attached to the frame that only opened in towards the robot in order to let cubes in from all directions but not out. The flaps were pushed open by the motionless cubes when the robot drove over them. The frame was built in 2 parts: the top frame, which was attached to the robot, and the square bottom frame, which held the flaps. The square frame was lower to the floor to reduce the risk of the flaps bending and letting cubes out. Figure 1 shows the completed robot. Appendix C shows our CAD models and cost calculations.

The robot had 2 motors, one for each wheel. We used a color sensor to detect when we reached the center of the board, which was where the cubes were. We had 2 QTI sensors which were used to detect the black border. An ultrasonic sensor was used to detect the opposing robot. We decided to use the breadboard from one of our kits in order to help prevent loose connections and provide more space for wiring. Appendix B shows the full circuit diagram.

We implemented 5 driving functions, which made the robot drive forward, drive backward, turn right, turn left, and stop. The forward driving function used software PWM to prevent the robot from drifting. We did not implement PWM for the other driving functions because they were only used for short bursts of movement. The `avoid_border()` function read the QTI sensor input and acted accordingly in order to prevent the robot from driving out of bounds. We also wrote multiple functions to implement the ultrasonic sensor and color sensor, and used interrupts for both. During Phase 1 of the competition behavior code, the robot drove forward until it reached the color border. If 10 seconds passed and the robot did not reach the color border, this meant that there was interference by the opposing robot and our robot would try to maneuver around it to reach the middle of the board. During Phase 2, the robot used the ultrasonic sensor to sense the opposing robot and turned accordingly. During Phase 3, the robot drove around the board, collecting more cubes and avoiding the black border. Appendix D summarizes the competition behavior in a flowchart. Appendix E shows the full competition code.

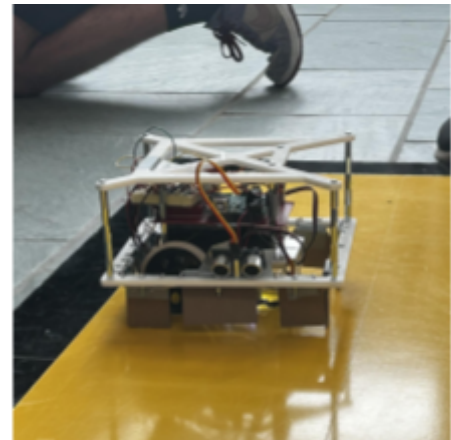


Figure 1. Completed robot. A white acrylic frame holds the cardboard flaps and the ultrasonic sensor. The color sensor and 2 QTI sensors are mounted to the bottom of the robot and wired to the Arduino.

During competition, the robot would ideally drive forward until it detected the color border. It would collect the middle third of the cubes and then use the ultrasonic sensor to see if the opposing robot was on its right side. Then, it would turn based on the sensor input and collect another third of the cubes. After that, it would drive around the board for the rest of the round, protecting its cubes and avoiding the opposing robot. With two thirds of the cubes collected, our robot would have the majority and therefore would win.

Design Process Reflection

Our initial design process consisted of sketches and brainstorming on what we wanted the robot to do from a mechanical, electrical, and software standpoint. Mechanically, our design changed from a circular frame to a square frame in order to utilize all of the space in the 8 inch by 8 inch initial bounding box. We also decided to use cardboard flaps with aluminum wire hinges for a cheaper build than laser cutting or 3D printing the one way doors. Additionally, the space we had under our robot to hold blocks was much smaller than we anticipated. This roadblock led to the choice of turning the robot around so the wheels were in the back, which allowed for more space for cubes to enter.

Electrically, we had to tape on a larger breadboard and rewire our sensors due to lack of space. We also taped down many wires to help prevent loose connections and replaced a few broken components. During Milestone 2, we had to replace one of our motors because the wires were too short and frayed. Also, during Milestone 3, one of our QTI sensors was constantly reading high no matter what, so we had to replace it. We were able to confirm this was a component issue by switching the wiring of the two sensors and seeing that the issue followed the sensor, not the wiring.

Our original algorithm had the robot stop in a corner after going to the middle to collect cubes. We decided to make it continuously drive to try and collect more cubes. This change was inspired by the fact that our robot kept collecting cubes while it drove and let almost none out. We also fine tuned the delays in the code to ensure that the robot turned exactly the amount we wanted to or drove for the correct amount of time. In doing so, we found that one of our motors was still stronger than the other, so we implemented software PWM to ensure that the robot drove in straight lines.

Competition Analysis

Overall, our robot performed well. Our team placed second in our group of 8, advancing us to the tournament. Unfortunately, we lost our first round.

The flaps did a good job holding the cubes in and not letting them out. Also, the robot's path worked very well and the robot consistently collected a lot of cubes. However, we consistently had issues with loose connections, particularly the QTI wires. Also, the ultrasonic sensor was ultimately not used at all because the opposing robot had to be positioned almost

exactly to the right of our robot and at the right time in order to be sensed. Another issue we faced is that the 8 inch by 8 inch initial bounding box was smaller than expected and the robot chassis took up a large portion of it. This caused cubes to get stuck in the wheels.

During the competition, there were more collisions with opposing robots than expected. This made our design work better than expected because our robot was able to collect cubes even while it was being pushed because the robot could collect cubes from all directions. This also meant that our robot got into many pushing matches with other robots. Fortunately, our design made our robot very stable, meaning it did not tip over and often won those matches. Also, sometimes the timer meant to get the robot to the color border in case of interference went off too early, which was unexpected because that did not happen during testing.

The robot's main strength was that it could collect cubes from all directions, even while being pushed. It also held the cubes very well once they were collected. However, our robot could only hold a maximum of 6 cubes on each side, or 12 total. Often, all of the collected cubes were concentrated on one side. Also, our robot's complex design meant that there were many places for small errors or loose connections. This meant that it was difficult to debug, particularly between competition rounds.

Conclusions

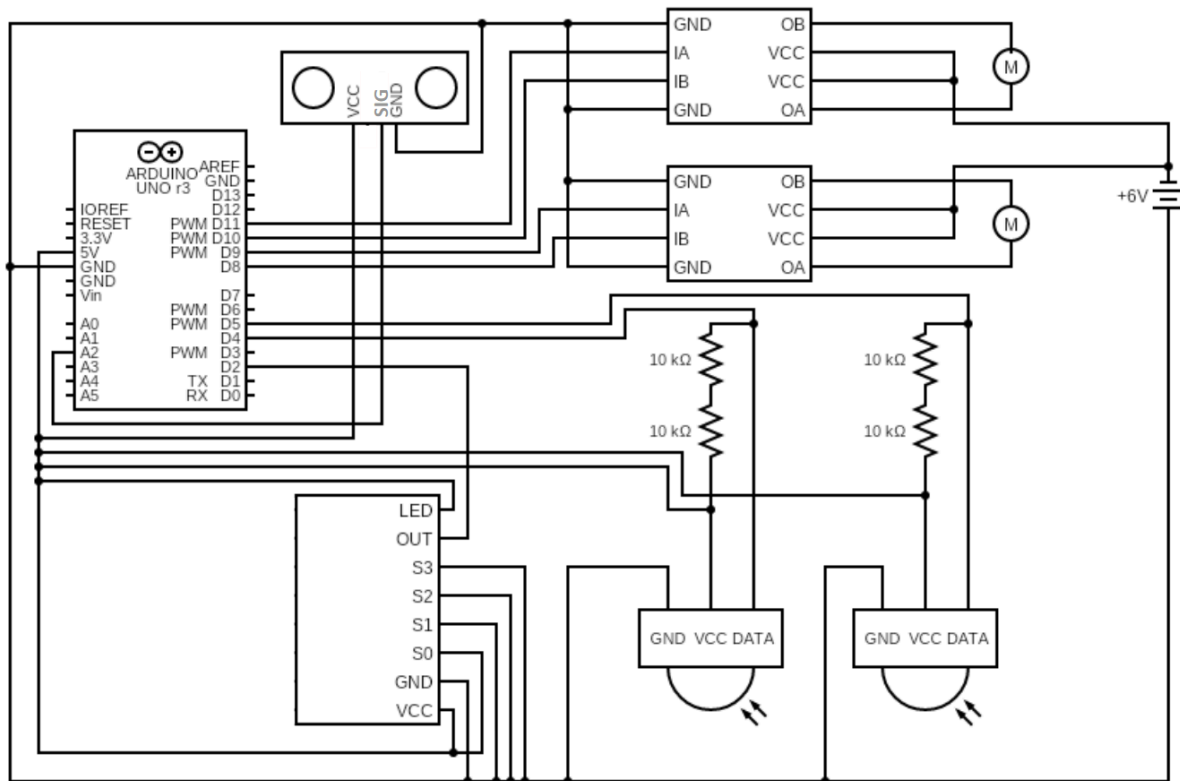
There are several parts of our design and wiring that we would change. Loose connections caused many of the issues we faced, and being more intentional with how we oriented our wires and where we placed the battery would have prevented a lot of them. Purchasing screw gate terminals would also have been useful to make our breadboard connections very secure. If the battery had been in a more accessible spot, especially toward the front, we would not have had wires popping out every time we swapped the battery or turned the robot on. We also should have added another support to the frame. With only three points of support, the structure was flimsy and more likely to bend or break under stress. Another change we could have made was to redesign the chassis to allow for more space to hold the collected cubes.

Based on our experience, we encourage future students to plan ahead and order materials early. We missed the order window for larger wheels and were not able to come up with another plan to make the robot faster in time. Another important takeaway is that the most successful robots were both fast and able to stay within the boundaries. When our QTI wiring was secure, our robot stayed in bounds and collected more cubes, which gave us an advantage over teams whose robots drove out. Speed is helpful for gathering cubes quickly, but it is just as important to make sure the cubes do not fall out while the robot is moving. Finally, wiring neatly and securely is essential. Loose connections made competition day much harder for us, since we had to troubleshoot after almost every round.

Appendix A: Bill of Materials

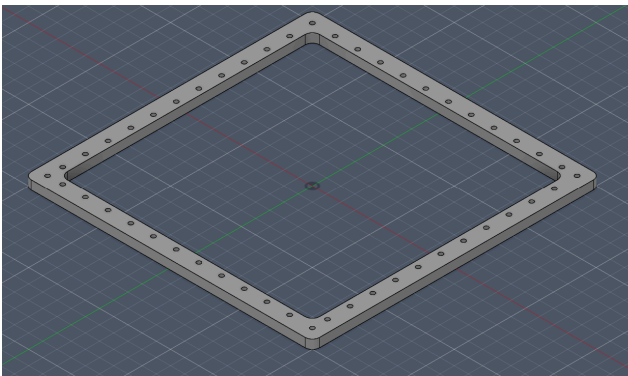
Part Number	Name	Description	Type of Part	Supplier	Supplier Part Number	Quantity	Unit Cost	Total Cost
1	Aluminum Wire	0.081" Diameter (-0.002" to 0.002" Tolerance)	Purchased	McMaster	8904K96	1	\$10.10	\$10.10
2	Top Frame		Laser cut	RPL		1	\$9.30	\$9.30
3	Square Frame		Laser cut	RPL		1	\$9.35	\$9.35
4	Square Frame	Not used	Laser cut	RPL		1	\$8.54	\$8.54
5	Cardboard						\$0.00	\$0.00
							Total:	\$37.29

Appendix B: Circuit Diagram



Appendix C: CAD Files and Drawings

Square Frame

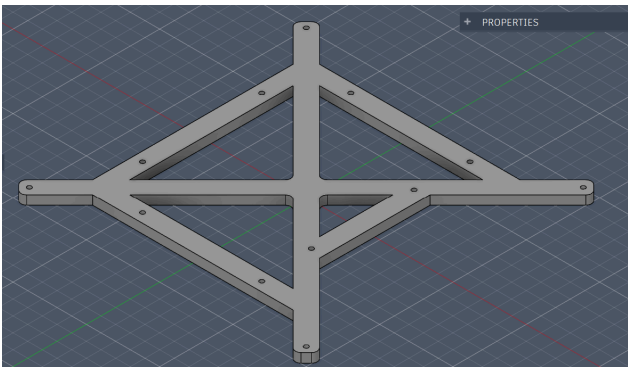


Mass	2.485 ouncemass
Volume	3.619 in ³
Density	0.687 ouncemass / in ³
Area	48.229 in ²
World X,Y,Z	0.00 in, 0.00 in, 0.00 in
Center of Mass	0.00 in, 0.002 in, 0.125 in

▼ Selection 1	
Face selected	
Area	14.474 in ²
Loop Length	77.124 in

Part #3 Cost Calculation: $\$0.05(77.124)+\$0.50+\$5=\9.35

Top Frame

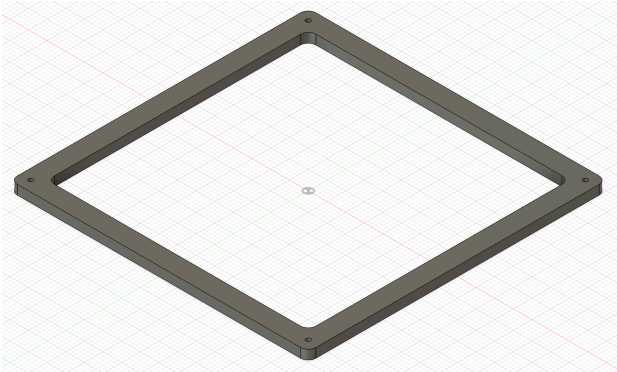


Mass	3.274 ouncemass
Volume	4.768 in ³
Density	0.687 ouncemass / in ³
Area	57.127 in ²
World X,Y,Z	0.00 in, 0.00 in, 0.00 in
Center of Mass	-0.254 in, -1.131E-05 in, 0....

▼ Selection 1	
Face selected	
Area	19.073 in ²
Loop Length	75.92 in

Part #2 Cost Calculation: $\$0.05(75.92)+\$0.50+\$5=\9.30

Square Frame - Not Used

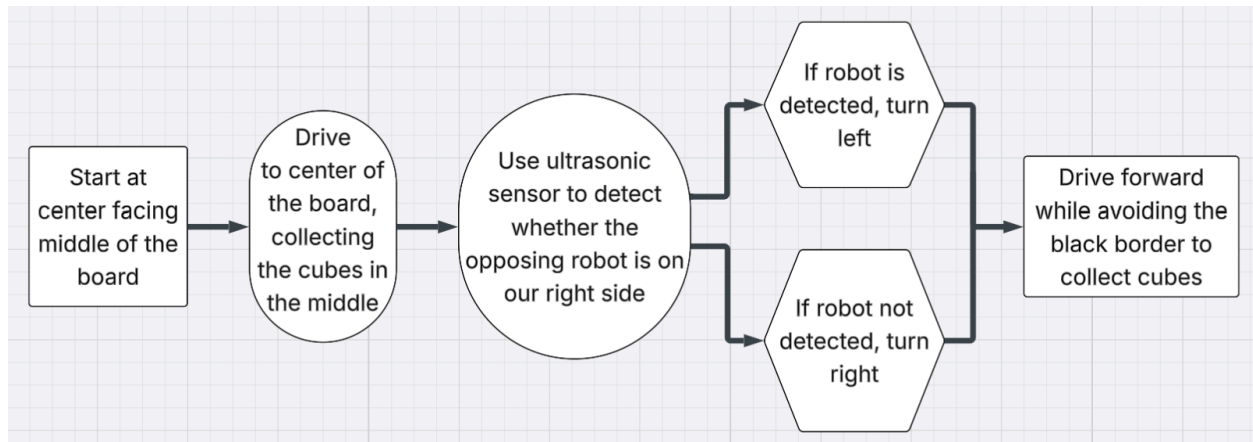


▼ Physical	
Mass	16.966 ouncemass
Volume	3.739 in ³
Density	4.538 ouncemass / in ³
Area	45.112 in ²
World X,Y,Z	0.00 in, 0.00 in, 0.00 in
Center of Mass	0.00 in, 0.00 in, 0.125 in

▼ Selection 1	
Face selected	
Area	14.956 in ²
Loop Length	60.797 in

Part #4 Cost Calculation: $\$0.05(60.797) + \$0.50 + \$5 = \8.54

Appendix D: Flowchart



Appendix E: Code

```
//----- PIN ASSIGNMENTS -----
// 2: Color sensor output
// 4: Right QTI sensor
// 5: Left QTI sensor
// 8: Left motor forward
// 9: Left motor backward
// 10: Right motor backward
// 11: Right motor forward
// A1: Ultrasonic sensor (trigger + echo)

#define PING_PIN PC1

// --- Global Variables ---

volatile unsigned int period = 0;
volatile unsigned int timer = 0;
volatile uint32_t timer2_overflows = 0;

// PWM calibration values
int leftDuty = 100;
int rightDuty = 80;

//----- INTERRUPT FUNCTIONS
-----
// --- ISR: an interrupt function that resets and reads the timer value
// COLOR SENSOR
```

```

ISR(PCINT2_vect)
{
    if (PIND & (1 << PIND2)) {
        TCNT1 = 0;
    }
    else {
        timer = TCNT1;
    }
}

// ISR: Increments overflow counter every time Timer2 rolls over.
// Used to extend Timer2 from 8-bit to 32-bit timing resolution.
// ULTRASONIC SENSOR
ISR(TIMER2_OVF_vect) {
    timer2_overflows++;
}

// ----- FUNCTIONS
-----

// --- initColor: initializes the interrupt and timer
// COLOR SENSOR
void initColor()
{
    DDRD &= ~( (1 << DDD2) | (1 << DDD3) );
    DDRD &= ~0b00110000; // pins 4 and 5 inputs for QTI
    DDRB |= 0b00001111;

    PCICR |= (1 << PCIE2);
    PCMSK2 &= ~(1 << PCINT18);

    TCCR1A = 0;
    TCCR1B = (1 << CS10);
    TCNT1 = 0;

    sei();
}

// --- getColor
// COLOR SENSOR
int getColor()

```

```

{
    PCMSK2 |= (1 << PCINT18);
    _delay_ms(10);
    PCMSK2 &= ~(1 << PCINT18);

    period = timer / 8;
    return period;
}

//Initializes Timer2 and the ultrasonic sensor pin.
// ULTRASONIC SENSOR
void initUltrasonic(void) {
    // Timer2 normal mode
    TCCR2A = 0;
    TCCR2B = 0;

    // Prescaler = 8
    // 16 MHz / 8 = 2 MHz, so each tick = 0.5 us
    TCCR2B |= 0b00000010;

    // Enable Timer2 overflow interrupt
    TIMSK2 |= 0b00000001;

    // Start with PING pin as input
    DDRC &= ~(1 << PING_PIN);

    sei();
}

//Safely resets Timer2 count and overflow counter
// ULTRASONIC SENSOR
void resetTimer2Count(void) {
    uint8_t oldSREG = SREG;
    cli();

    TCNT2 = 0;
    timer2_overflows = 0;
    TIFR2 |= 0b00000001; // clear overflow flag

    SREG = oldSREG;
}

```

```

}

//Returns total elapsed Timer2 ticks (including overflows)
//Each tick = 0.5 microseconds (with prescaler = 8)
// ULTRASONIC SENSOR
uint32_t getTimer2Ticks(void) {
    uint8_t oldSREG = SREG;
    cli();

    uint32_t ticks = (timer2_overflows * 256UL) + TCNT2;

    SREG = oldSREG;
    return ticks;
}

//Sends a trigger pulse to the ultrasonic sensor and measures echo
duration
//Returns pulse duration in microseconds
// ULTRASONIC SENSOR
unsigned int getUltrasonicPulseUS(void) {

    // Send trigger pulse
    DDRC |= (1 << PING_PIN);      // output

    PORTC &= ~(1 << PING_PIN);
    _delay_us(2);

    PORTC |= (1 << PING_PIN);
    _delay_us(5);

    PORTC &= ~(1 << PING_PIN);

    DDRC &= ~(1 << PING_PIN);      // input

    // Wait for echo to go HIGH
    resetTimer2Count();
    while (!(PINC & (1 << PING_PIN))) {
        if (getTimer2Ticks() > 60000UL) {
            return 0;
        }
    }
}

```

```

    }

    // Measure how long echo stays HIGH
    resetTimer2Count();
    while (PINC & (1 << PING_PIN)) {
        if (getTimer2Ticks() > 60000UL) {
            return 0;
        }
    }

    uint32_t ticks = getTimer2Ticks();

    // each tick = 0.5 us
    return ticks / 2;
}

//Converts ultrasonic pulse duration into distance in inches
//Returns distance in inches
// ULTRASONIC SENSOR
unsigned int getDistanceInches(void) {
    unsigned int pulseUS = getUltrasonicPulseUS();

    if (pulseUS == 0) {
        return 999;    // no object / timeout
    }

    return pulseUS / 148;
}

//Determines if an object (e.g., opposing robot) is within a given
distance
//Returns true if object is within threshold distance, false otherwise
//ULTRASONIC SENSOR
bool robotDetected(unsigned int thresholdInches) {
    unsigned int distance = getDistanceInches();

    return distance <= thresholdInches;
}

// driving functions

```

```

// --- Software PWM forward driving ---
void drive_forward_pwm(int duration_ms){
    int elapsed = 0;

    while(elapsed < duration_ms){

        // Start each PWM cycle clean: clear all motor control pins
        PORTB = PORTB & 0b11110000;

        // Turn both motors forward
        PORTB = (PORTB & 0b11110000) | 0b00000110;

        for(int t = 0; t < 10; t++){

            if(t >= rightDuty / 10){
                // turn off left motor: clear BOTH left motor pins PB0 and PB1
                PORTB &= 0b11111100;
            }

            if(t >= leftDuty / 10){
                // turn off right motor: clear BOTH right motor pins PB2 and PB3
                PORTB &= 0b11110011;
            }

            _delay_ms(1);
        }

        elapsed += 10;
    }

    // End cleanly
    PORTB = PORTB & 0b11110000;
}

void drive_backward(void){
    PORTB = (PORTB & 0b11110000) | 0b00001001;
}

void turn_left(void){

```

```

    PORTB = (PORTB & 0b11110000) | 0b00000101;
}

void turn_right(void) {
    PORTB = (PORTB & 0b11110000) | 0b00001010;
}

void stop(void) {
    PORTB = (PORTB & 0b11110000) | 0b00000000;
}

//checks for black border and takes action
//QTI SENSOR
void avoid_border(void) {
    //variable with just sensor input (pins 4 and 5)
    unsigned char sensors = PIND & 0b00110000;
    //if robot hits border head on, back up and turn around
    if (sensors == 0b00110000) { //both QTIs high
        stop();
        _delay_ms(10);
        drive_backward();
        _delay_ms(200);
        stop();
        turn_right();
        _delay_ms(1200); //180 degrees
    }
    //if robot hits border on left, turn right away from the wall
    else if (sensors == 0b00100000) { //only pin 5 high
        turn_right();
        _delay_ms(200);
    }
    //if robot hits border on right, turn left away from the wall
    else if (sensors == 0b00010000) { //only pin 4 high
        turn_left();
        _delay_ms(200);
    }
    drive_forward_pwm(100);
}

```

```

// ----- PHASE 1: DRIVE FORWARD TO COLOR
BORDER (HOPEFULLY) -----

int main(void) {

    //initialize stuff
    init();
    initColor();
    initUltrasonic();

    //find starting color
    period = getColor();
    bool blue = period > 160;

    //started on blue and still on blue OR started on yellow and still on
yellow
    bool hit_color_border = !((blue && period > 160) || (!blue && period <
160));
    //loop iteration count
    int i = 0;
    int max_time = 10; //seconds

    //while not hit color border check color and move forward
while(!hit_color_border) {
    // drive forward
    drive_forward_pwm(100);

    // call getColor function to check color
    period = getColor();

    //check to make sure we didn't run into the black border
    avoid_border();

    //update loop conditions
    hit_color_border = !((blue && period > 160) || (!blue && period <
160));
    i++;

    //if max time has passed, there was interference from the other robot
    //detect if the other robot is to the side and if not try to go around
that way towards the color border

```

```

    if(i >= max_time*6) {
        stop();
        _delay_ms(10);
        drive_backward();
        _delay_ms(400);
        if(robotDetected(8)) {
            turn_right();
            _delay_ms(650);
            drive_forward_pwm(600);
            turn_left();
            _delay_ms(750);
        }
        else {
            turn_left();
            _delay_ms(750);
            drive_forward_pwm(600);
            turn_right();
            _delay_ms(650);
        }
        i = 0;
    }
}

//----- PHASE 2: DETECT OPPOSING ROBOT
-----

if(robotDetected(10)) {
    turn_right();
    _delay_ms(650);
}
else {
    turn_left();
    _delay_ms(750);
}

// ----- PHASE 3: COLLECT & PROTECT THE CUBES
-----

//drive around the border
while(1) {
    drive_forward_pwm(100);
    avoid_border();
}
}

```

Code References and Credits:

1. ChatGPT was used to help code the PWM and the ultrasonic sensor.
2. I referenced the Pre-lab 4 code to code the color sensor.